

¿Qué caracteriza a la programación imperativa y en qué se diferencia de la programación declarativa?

La programación Imperativa describe cómo se resuelve un problema: El programador especifica paso a paso las instrucciones que la computadora debe seguir para lograr el resultado

Está basada en comandos y estados: Utiliza estructuras de control (bucles, condicionales) y modifica el estado del programa mediante asignaciones de variables.

Ejemplo:

```
numeros = [1, 2, 3, 4]
suma = 0
for num in numeros:
    suma += num
print(suma)
```

1) Verdadero o Falso: La programación imperativa se basa en describir qué se quiere lograr, no cómo.

Falso, la programación imperativa se trata de ordenar a la computadora cómo debe hacer, no lo que quiero lograr.

2) ¿Qué estructuras básicas componen la programación imperativa?

Las estructuras básicas de la programación imperativa son:

Variables, Condicionales, Bucles, Funciones, Estructuras de Datos Básicas como (Arrays, Diccionarios, Tuplas) y manipulación de Estado

```
# Código con errores para corregir
# Código con mal uso del bucle y sin claridad
```

```
"""
nums = [1,2,3,4]
for i in nums:
    if i % 2 ==0:
        i = i * 2
        print(i)
"""
```

```
#Corregir para que solo los pares se dupliquen y la salida sea una
nueva lista
# Version corregida
```

```
numeros = [1,2,3,4]
nueva_lista = []
for i in numeros:
    if i % 2 ==0:
        nueva_lista.append(i * 2)
        print(nueva_lista)
```

```
# Crea una función que reciba una lista de números y devuelva otra
lista con solo los números pares multiplicados por 3, usando solo
estructuras imperativas (no comprensiones ni funciones como map o
filter)
```

```
numeros = [1,2,3,4,5,6,7,8,9]
nueva_lista = []

def multiplica(numeros):
    for i in numeros:
        if i % 2 == 0:
            nueva_lista.append(i*3)
```

```
multiplica(numeros)
print(nueva_lista)
```

💎 Reflexión individual

¿Qué diferencias notaste entre escribir código imperativo y usar comprensiones o funciones como map o filter?

Con el enfoque declarativo con list comprensiones o map/filter

Se describe que quieres hacer (no cómo hacerlo)

Hay menos código: Más conciso y expresivo

Inmutabilidad: No modificas listas existentes, creas nuevas.

List Comprehension

```
nueva_lista = [x * 3 for x in numeros if x % 2 == 0]
print(nueva_lista) # Output: [6, 12, 18, 24]
```

Map + Filter

```
nueva_lista = list(map(lambda x: x * 3, filter(lambda x: x % 2 == 0,
numeros)))
print(nueva_lista) # Output: [6, 12, 18, 24]
```

De esta forma suele ser más rápido en rendimiento en casos grandes.

Uso de ambos mundos Imperativo y declarativo

```
numeros = [1,2,3,4,5,6,7,8,9]
```

```
def es_par(x):
    return x % 2 == 0
```

```
nueva_lista = [x * 3 for x in numeros if es_par(x)]
print (nueva_lista)
```

Uso declarativo

```
numeros = [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
nueva_lista = [x * 3 for x in numeros if x % 2 == 0]
print(nueva_lista) # Output: [6, 12, 18, 24]
```